

Module 1 — Fondamentaux des Large Language Models

LLM, Prompt, RAG & Agents IA

Bassem Ben Hamed & Nesrine Trabelsi

Bootcamp

Avril 2026

Partie 1 — Introduction à l'IA générative

- ➊ Panorama de l'IA
- ➋ Types d'apprentissage
- ➌ IA discriminative vs IA générative
- ➍ Cas d'usage (discriminatif puis génératif)
- ➎ Métriques d'évaluation
- ➏ Des règles aux LLM : bref historique
- ➐ Écosystème LLM 2026

Partie 2 — Le Transformer pas à pas

- ➑ Tokenisation
- ➒ Architecture Encodeur-Décodeur (RNN → Tr.)
- ➓ Vue d'ensemble (*Vaswani 2017*)
- ➑ Étape 1 — Embeddings des tokens
- ➒ Étape 2 — Encodage positionnel
- ➓ Étape 3 — Scaled Dot-Product Attention
- ➑ Étape 4 — Multi-Head Attention
- ➒ Étape 5 — FFN + Add & Norm (bloc encodeur)
- ➓ Étape 6 — Décodeur : Masked Self-Attention
- ➑ Étape 7 — Décodeur : Cross-Attention
- ➒ Prédiction token-par-token & décodage
- ➓ Préparation TP1

Objectifs d'apprentissage

À l'issue de ce module, vous serez capable de :

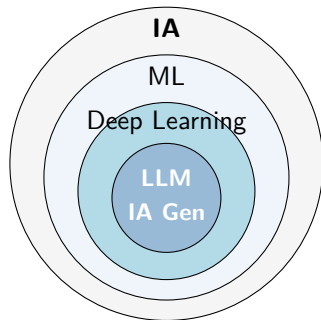
- 1 **Situer** les LLM dans le champ plus large de l'IA et distinguer les approches discriminative et générative.
- 2 **Distinguer** les trois grands types d'apprentissage : supervisé, non supervisé, par renforcement.
- 3 **Expliquer** les briques fondamentales d'un LLM : tokenisation, embeddings, attention (Q, K, V).
- 4 **Décrire** le mécanisme de prédiction token-par-token et l'impact des paramètres de décodage.
- 5 **Évaluer** un modèle avec les métriques adaptées (accuracy, F1 pour le discriminatif ; BLEU, ROUGE, perplexité pour le génératif).
- 6 **Choisir** un modèle adapté à un cas d'usage selon critères coût / qualité / conformité.

Partie 1

Introduction à l'IA générative

Où se situe le NLP dans le paysage de l'IA ?

- **IA** — champ qui vise à faire effectuer à des machines des tâches qui nécessitent de l'intelligence.
- **ML** — sous-ensemble de l'IA : apprendre des données plutôt que de coder des règles.
- **Deep Learning** — ML à base de réseaux de neurones profonds.
- **NLP** — traitement automatique du langage.
- **LLM** — NLP à large échelle, basé sur Transformers.
- **IA générative** — produit du contenu (texte, image, code) plutôt que classifier.



Les trois grands types d'apprentissage

Supervisé

Apprend depuis (X, y) : on connaît la **réponse attendue**.

- Classification (fraude oui/non)
- Régression (montant prédit)

Algos : régression logistique, SVM, Random Forest, XGBoost, réseaux de neurones.

Non supervisé

Apprend depuis X seul : aucune étiquette, on cherche une **structure cachée**.

- Clustering (segmentation client)
- Réduction de dimension (PCA, UMAP)
- Détection d'anomalies

Algos : K-means, DBSCAN, autoencodeurs.

Par renforcement

Un **agent** interagit avec un environnement et apprend par **essai / récompense**.

- Jeux (AlphaGo)
- Robotique
- Trading algorithmique
- RLHF (alignement des LLM)

Algos : Q-learning, PPO, DQN.

Cas particulier : auto-supervisé

Les LLM s'entraînent en **prédissant le token suivant** à partir de texte brut — pas d'étiquette manuelle, mais structure supervisée extraite automatiquement du corpus.

IA discriminative vs IA générative

Discriminative — $P(y | x)$

- Apprend une **frontière** de décision.
- Entrée $x \rightarrow$ étiquette y .
- Exemples : détection de fraude, scoring crédit, classification de sentiment.
- Outils : régression logistique, SVM, Random Forest, XGBoost, CNN.

Générative — $P(x)$ ou $P(x, y)$

- Apprend la **distribution** des données.
- Peut **échantillonner** / produire du nouveau contenu.
- Exemples : ChatGPT, Claude, Midjourney, Copilot.
- Outils : Transformers, Diffusion Models, VAE / GAN.

À retenir

Les LLM sont **génératifs** : ils produisent le token suivant le plus probable, mais peuvent aussi classifier, extraire, résumer.

Cas d'usage IA — discriminatif (ML / DL classique)

Vision & reconnaissance (CNN / DL)

- **Classification d'images** (ImageNet, médical).
- **Détection d'objets** (YOLO, Faster R-CNN).
- **Reconnaissance OCR** de documents.
- **Segmentation d'images** (U-Net).

NLP discriminatif (Transformers encoder-only)

- **Classification** de tickets / sentiments.
- **NER** (extraction d'entités nommées).
- **Détection de spam** / toxicité.

Données tabulaires (ML classique)

- **Détection d'anomalies** (Isolation Forest, AutoEncoder).
- **Scoring & churn** (XGBoost, LightGBM).
- **Forecasting** de séries temporelles (Prophet, ARIMA).
- **Clustering** (K-means, DBSCAN, HDBSCAN).

Recommandation & matching

- **Systèmes de reco** (matrix factorization, two-tower).
- **Recherche sémantique** via embeddings (ANN).

Cas d'usage IA — génératif (LLM, RAG, agents, fine-tuning)

Génération de texte / code (LLM seuls)

- **Chatbot & assistant conversationnel.**
- **Génération de code** (Copilot, Cursor, Claude Code).
- **Résumé, traduction, reformulation.**
- **Génération de SQL** depuis langage naturel.

RAG (LLM + base de connaissances)

- **Q&R** sur documentation interne / wiki.
- **AI Research Assistant** (papiers, biblio).
- **Support technique** sur knowledge base.
- **Onboarding & assistant** nouveaux collaborateurs.

Agents IA (LLM + outils)

- **Agent ReAct** : RAG + web search + APIs.
- **Code agent** : lit le repo, lance les tests, propose un patch.
- **Adaptive RAG** (LangGraph) : self-eval + fallback web.
- **Workflows no-code** (n8n, Make).

Fine-tuning & adaptation

- **LoRA / QLoRA** pour adapter style & vocabulaire.
- **Instruction-tuning** sur dataset métier.
- **RLHF / DPO** pour alignement comportement.

Métriques d'évaluation — IA discriminative

Matrice de confusion :

	Préd. +	Préd. -
Réel +	TP	FN
Réel -	FP	TN

Accuracy

$$\text{Acc} = \frac{TP + TN}{TP + TN + FP + FN}$$

Trompeuse si classes déséquilibrées (fraude $\approx$ 0,1%).

Precision — qualité des positifs

$$P = \frac{TP}{TP + FP}$$

« Sur les positifs prédits, combien corrects ? »

Recall — couverture des positifs

$$R = \frac{TP}{TP + FN}$$

« Des positifs réels, combien détectés ? »

F1-score — moyenne harmonique P / R

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Référence en classification déséquilibrée.

Métriques d'évaluation — IA générative

BLEU (traduction) — précision sur n -grammes

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

p_n : précision n -gr. ; $w_n = 1/N$; $\text{BP} = \min(1, e^{1-r/c})$ pénalise les sorties courtes.

ROUGE-N (résumé) — recall n -grammes

$$\text{ROUGE-N} = \frac{\sum \text{Count}_{\text{match}}(n\text{-gr})}{\sum \text{Count}(n\text{-gr})}$$

Variantes : ROUGE-1, ROUGE-2, ROUGE-L (LCS).

Perplexité — qualité intrinsèque

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_i \log P(w_i \mid w_{<i}) \right)$$

Plus c'est **bas**, mieux c'est. $\text{PPL} = 50 \Leftrightarrow 50$ mots possibles.

Limites

BLEU / ROUGE pénalisent les paraphrases. Évaluation humaine ou **LLM-as-a-judge** restent indispensables.

Des règles aux LLM : 70 ans en 1 slide

Période	Paradigme	Caractéristique
1950–1980	Systèmes à base de règles	Grammaires manuelles, peu généralisables
1990–2010	NLP statistique (n-grammes, HMM)	Probabilités locales, Bag-of-Words, TF-IDF
2013–2017	Word embeddings (Word2Vec, GloVe)	Sens capturé dans des vecteurs denses
2017	Transformers (« Attention is all you need »)	Architecture qui change tout : parallélisable, attention globale
2018–2020	BERT, GPT-2, GPT-3	Pré-entraînement massif + fine-tuning
2022–2026	ChatGPT, Claude, GPT-4o, Llama, Mistral	LLM grand public, capacités émergentes, multimodalité

Modèle	Éditeur	Accès	Spécificité
GPT-5.x	OpenAI / Azure	API, Copilot	Polyvalent, multimodal
Claude 4.x	Anthropic	API, AWS Bedrock	Long contexte, raisonnement
Gemini 3.x	Google	API Vertex AI	Multimodal natif
Mistral / Mixtral	Mistral AI	API, self-host	Européen, open-weight
Llama 3.x / 4.x	Meta	Open-source	Self-host, fine-tunable
Phi-4	Microsoft	Azure, local	Petit modèle efficace

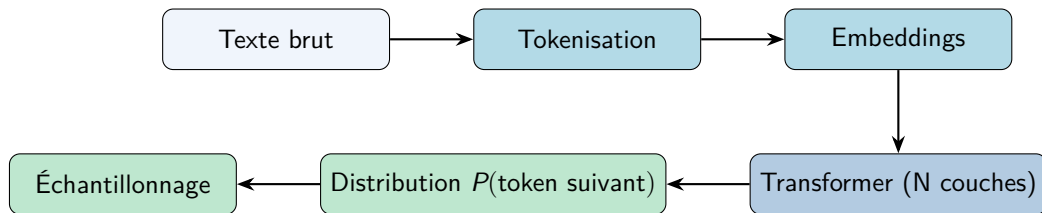
Critères de choix en contexte bancaire

- **Souveraineté** : données sensibles → Azure OpenAI ou hébergement interne.
- **Coût / latence** : petit modèle pour classification, grand pour raisonnement.
- **Contexte** : documents longs → Claude ou GPT-4 long-context.

Partie 2

Les LLM : comprendre la brique

De quoi est fait un LLM ? Vue d'ensemble



- Le LLM est une **fonction** : il prend du texte, il renvoie une distribution de probabilités sur le prochain token.
- **Répété en boucle** : chaque token généré est ré-injecté en entrée.
- Aucune « compréhension » au sens humain : il prédit statistiquement la suite la plus plausible.

Tokenisation : découper le texte en unités

Définition

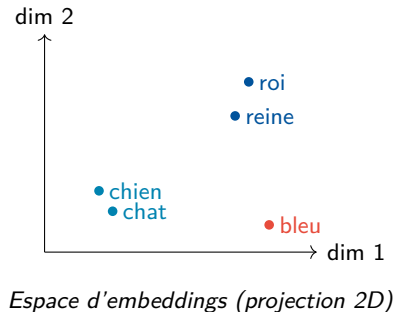
La tokenisation transforme une chaîne de caractères en une séquence d'entiers (identifiants) que le modèle peut manipuler.

Granularité	Exemple sur « J'analyse les flux »
Caractère	J ' a n a l y s e ...
Mot	J'analyse les flux
Sous-mot (BPE)	J' analy se les flux (<i>standard en LLM</i>)

- Les LLM modernes utilisent des tokenizers **sous-mots** : BPE, WordPiece, SentencePiece.
- Règle de pouce : **1 token** $\approx$ **0,75 mot** en anglais, $\approx$ **0,5 mot** en français.
- Le coût et la limite de contexte se comptent en **tokens**, pas en mots.

Embeddings : représenter le sens par des vecteurs

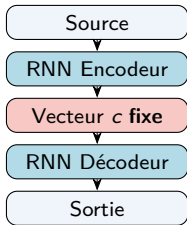
- Chaque token est projeté dans un vecteur dense de dimension d ($d = 768, 4096, \dots$).
- Des tokens **sémantiquement proches** donnent des vecteurs proches (distance cosinus faible).
- Propriété célèbre :
 $\text{roi} - \text{homme} + \text{femme} \approx \text{reine}$
- Ces embeddings sont **appris** pendant l'entraînement, puis réutilisables (recherche sémantique, RAG, clustering).



Architecture Encodeur-Décodeur : du RNN au Transformer

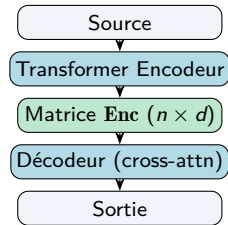
Principe seq2seq : un **encodeur** compresse l'entrée en représentation vectorielle, un **décodeur** la consomme pour générer la sortie token par token.

RNN seq2seq (Sutskever 2014)



Toute la phrase dans **un seul vecteur** → goulot d'étranglement, séquentiel.

Transformer seq2seq (Vaswani 2017)

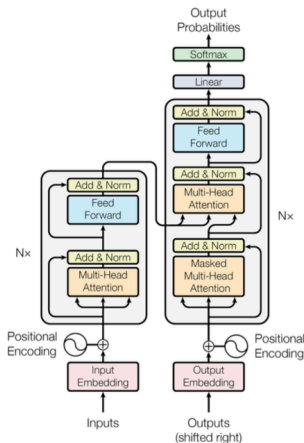


Un vecteur par token source, interrogé via **cross-attention**. Parallélisable.

Lien avec le RAG (jour 2)

Le « vector store » d'un système RAG joue un rôle analogue : un index de vecteurs interrogé par le LLM avant génération.

Le Transformer : vue d'ensemble



Vaswani et al., 2017

Deux pôles

Encodeur (gauche) lit la phrase source. **Décodeur** (droite) génère la cible token par token, en regardant la sortie de l'encodeur.

Fil rouge : « je suis étudiant » → « I am a student »

Exemple simplifié (calculs à la main) : $d_{\text{model}} = 4$ (dim. vecteurs), $h = 2$ (têtes parallèles), $d_k = d_v = 2$ (dim. clé-valeur), $d_{ff} = 8$ (couche cachée), $N = 1$ (couches empilées).

Pipeline en 6 étapes :

- 1 Embeddings des tokens
- 2 Encodage positionnel
- 3 Encodeur : Scaled dot-product attention
- 4 Multi-head + FFN + Add&Norm → **vector store contextuel**
- 5 Décodeur : Masked self-attention
- 6 Décodeur : Cross-attention + FFN + Add&Norm

Étape 1 — Embeddings des tokens

Idée

Chaque token (entier) est converti en un **vecteur dense** de dimension d_{model} via une matrice apprise $W_{\text{emb}} \in \mathbb{R}^{|V| \times d_{\text{model}}}$.

$$E(t) = W_{\text{emb}}[t] \times \sqrt{d_{\text{model}}}$$

- Indices de notre phrase : je=3, suis=4, étudiant=5.
- W_{emb} **apprise** : tokens proches en sens $\rightarrow$ vecteurs proches.
- Le facteur $\sqrt{d_{\text{model}}} = \sqrt{4} = 2$ met l'embedding à la même échelle que le PE (étape 2).
- Tailles réelles : $|V| \sim 50\text{k} - 200\text{k}$, $d_{\text{model}} = 512$ (papier), 4096 (Llama 70B)...

« je suis étudiant » ($d_{\text{model}} = 4$)

Lignes 3–5 de W_{emb} , multipliées par $\sqrt{4} = 2$:

$$\begin{pmatrix} 0.5 & 0.3 & -0.2 & 0.1 \\ 0.2 & -0.1 & 0.4 & 0.3 \\ -0.3 & 0.6 & 0.1 & -0.4 \end{pmatrix} \cdot 2 \rightarrow$$

$$X_{\text{emb}} = \begin{pmatrix} 1.0 & 0.6 & -0.4 & 0.2 \\ 0.4 & -0.2 & 0.8 & 0.6 \\ -0.6 & 1.2 & 0.2 & -0.8 \end{pmatrix}$$

Une ligne par token (« je », « suis », « étudiant »).

Étape 2 — Encodage positionnel

Pourquoi ?

Le Transformer regarde tous les tokens en parallèle — aucune notion d'ordre. Sans PE, « je suis étudiant » et « étudiant suis je » ont la même représentation.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

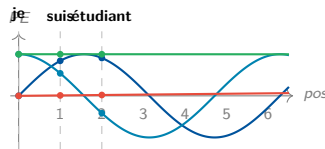
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Avec $d_{\text{model}}=4$, 4 dimensions :

- dim 0 = $\sin(pos/1)$ haute fréq.
- dim 1 = $\cos(pos/1)$ haute fréq.
- dim 2 = $\sin(pos/100)$ basse fréq.
- dim 3 = $\cos(pos/100)$ basse fréq.

3 tokens, 3 signatures uniques :

$$PE = \begin{pmatrix} 0.000 & 1.000 & 0.000 & 1.000 \\ 0.841 & 0.540 & 0.010 & 1.000 \\ 0.909 & -0.416 & 0.020 & 1.000 \end{pmatrix}$$



Entrée du Transformer

$X = X_{\text{emb}} + PE$: même forme, l'ordre est encodé.

Modèles modernes : **RoPE** / **ALiBi** (positions relatives, longs contextes).

Étape 3 — Encodeur : Scaled Dot-Product Attention

Q, K, V : trois rôles, un seul mécanisme

- **Q (Query)** = « ce que je cherche » — point de vue du token *actif*.
- **K (Key)** = « ce que je propose comme étiquette » — annonce des autres tokens.
- **V (Value)** = « ce que je transporte si on me sélectionne » — info à propager.

Analogie : la bibliothèque

- Q = ma question (« un livre sur l'astronomie ? »)
- K = étiquettes des rayons (« sciences », « astro », « histoire »)
- V = contenu des livres
- $\text{softmax}(QK^T / \sqrt{d_k}) \rightarrow$ « astro 90 %, sciences 8 %, histoire 2 % »
- Sortie : *mélange pondéré* des V .

En NLP : « la voiture...elle est à plat »

- Q d'« elle » = « pronom fém. sing., je cherche un antécédent ».
- K de « voiture » = « je suis nom fém. sing. ».
- Match $\rightarrow$ poids fort $\rightarrow$ « elle » récupère le V de « voiture ».

Calcul (3 étapes)

- 1 **Projeter** : $Q = XW^Q$, $K = XW^K$, $V = XW^V$
- 2 **Comparer** : $A = \text{softmax}(QK^T / \sqrt{d_k})$
- 3 **Mélanger** : $\text{Attention} = A \cdot V$

Le $\sqrt{d_k}$ stabilise : sinon QK^T explose en grande dim $\rightarrow$ softmax saturé, gradients ≈ 0 .

Tête 1 sur « je suis étudiant » ($d_k=2$)

Poids A_1 (lignes = *qui regarde*, colonnes = *qui est regardé*) :

$$A_1 = \begin{pmatrix} .318 & .350 & .331 \\ .361 & .344 & .295 \\ .335 & .340 & .325 \end{pmatrix} \begin{matrix} \leftarrow \text{je} \\ \leftarrow \text{suis} \\ \leftarrow \text{étud.} \end{matrix}$$

« je » regarde « suis » à 35 %. Distribution lissée (réseau peu entraîné).

Étape 4 — Multi-Head Attention (plusieurs regards en parallèle)

Idée

Une seule tête **moyenne** les relations. On lance h attentions en parallèle, chacune apprend un *type* de relation différent. On concatène et on projette.

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad \text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$$

Sur « je suis étudiant » ($h=2, d_v=2$)

Chaque tête sort 3×2 (3 tokens, d_v cols) :

$$\text{head}_1 = \begin{pmatrix} -.023 & .632 \\ -.038 & .652 \\ -.032 & .636 \end{pmatrix} \quad \text{head}_2 = \begin{pmatrix} .150 & -.100 \\ .220 & .050 \\ .180 & -.030 \end{pmatrix}$$

Concat 3×4 puis W^O (4×4) $\rightarrow$ retour à $d_{\text{model}} = 4$.

Dimensions usuelles : papier $d_{\text{model}} = 512$, $h = 8$, $d_k = 64$; modernes $h = 32-96$.

Règle : $h \cdot d_k = d_{\text{model}}$ (coût ≈ 1 seule tête).

Interprétation des têtes (BERTology, Anthropic)

- Tête A : **sujet** $\leftrightarrow$ **verbe**
- Tête B : **coréférence** (pronom $\leftrightarrow$ antécédent)
- Tête C : positions **locales** (préc./suiv.)
- Tête D : **sémantique** (synonymes, hyponymes)
- Tête E : **ponctuation** & balises

À retenir

Aucune tête n'est imposée : leurs rôles *émergent* à l'entraînement et s'observent *a posteriori*.

Étape 5 — Feed-Forward + Add & Norm = bloc Encodeur complet

Feed-Forward Network (FFN)

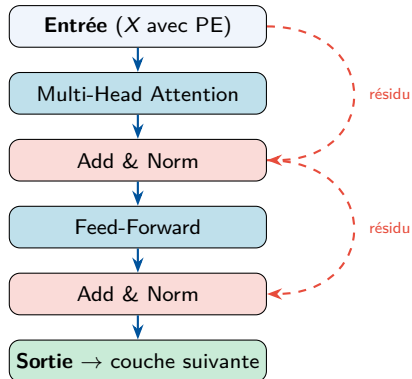
$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1) W_2 + b_2$$

- 2 couches linéaires + ReLU/GELU ; appliqué **indépendamment à chaque position**.
- **Non-linéarité** ; **~95 % des paramètres** y résident.
- $d_{\text{model}} \rightarrow d_{\text{ff}} \rightarrow d_{\text{model}}$ avec $d_{\text{ff}} = 4d_{\text{model}}$. Exemple : $4 \rightarrow 8 \rightarrow 4$.

Add & Norm (autour de chaque sous-couche)

$$y = \text{LayerNorm}(x + \text{SubLayer}(x))$$

- **Add** = résidu : x contourne la sous-couche $\rightarrow$ gradients préservés (ResNet).
- **Norm** = LayerNorm par token : $\frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \gamma + \beta$.



Empilement

$N = 6$ blocs encodeur (papier), 12–96 dans les LLMs modernes.

Sortie de l'encodeur : un « vector store » contextuel

Ce qui sort de l'encodeur après N blocs

Une matrice $\mathbf{Enc} \in \mathbb{R}^{n \times d_{\text{model}}}$: **une ligne par token source**, mais chaque ligne contient désormais **l'information de toute la phrase**.

Avant l'encodeur ($X_{\text{emb}} + PE$) : représentations *isolées* — chaque token connaît juste son sens et sa position.

Après l'encodeur (self-attention $\times N$) : chaque vecteur a **absorbé le contexte**.

- Le vecteur de « étudiant » « sait » qu'il suit « je suis ».
- Le vecteur de « suis » « sait » que son sujet est « je ».
- Polysémie résolue : « avocat » fruit vs. métier $\rightarrow$ deux $\mathbf{Enc}$ très différents selon la phrase.

Notre exemple (3×4)

$$\mathbf{Enc} = \begin{pmatrix} \text{enc}_{11} & \text{enc}_{12} & \text{enc}_{13} & \text{enc}_{14} \\ \text{enc}_{21} & \text{enc}_{22} & \text{enc}_{23} & \text{enc}_{24} \\ \text{enc}_{31} & \text{enc}_{32} & \text{enc}_{33} & \text{enc}_{34} \end{pmatrix} \begin{array}{l} \leftarrow \text{« je »} \\ \leftarrow \text{« suis »} \\ \leftarrow \text{« étudiant »} \end{array}$$

3 vecteurs *contextualisés* de dimension 4.

Analogie : vector store

$\mathbf{Enc}$ = base vectorielle indexée par token, calculée à la volée :

- **Indexation** : n vecteurs de dim d_{model} .
- **Recherche** : la cross-attention du décodeur l'interroge avec ses Q .
- **vs. RAG** : ici les vecteurs sont recalculés par phrase, pas persistés.

Rôle pour le décodeur

$\mathbf{Enc}$ devient les K et V de la **cross-attention** (étape 7). **Calculé une seule fois**, réutilisé à chaque pas auto-régressif.

Étape 6 — Décodeur : Masked Multi-Head Self-Attention

Pourquoi un masque ?

Le décodeur génère la cible **token par token, de gauche à droite** (auto-régressif). Quand il prédit le mot i , il **ne doit pas voir** les mots $i + 1, i + 2, \dots$ (sinon il triche pendant l'entraînement).

Masque causal

- On ajoute $-\infty$ aux positions « futur » de la matrice QK^T **avant** le softmax.
- $\text{softmax}(-\infty) = 0$: ces positions ont un poids nul.
- Tout le reste est identique à la self-attention de l'encodeur.

Pour « I am a student », en générant le 3^e token (« a ») :

- le décodeur voit « I, am, a » ;
- le masque cache « student » et « <eos> ».

Matrice de masque (causal)

$$\begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Triangulaire inférieure de zéros, $-\infty$ au-dessus de la diagonale.

Et après ce sous-bloc ?

Add & Norm appliqué → on passe à la cross-attention (étape 7).

Étape 7 — Décodeur : Cross-Attention + FFN + Add & Norm

Cross-attention : où le décodeur « regarde » la phrase source

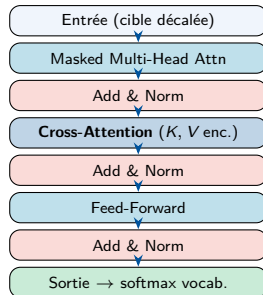
Même formule que l'attention, mais Q vient du **décodeur** et K, V de la **sortie de l'encodeur**.

$$\text{Cross-Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad Q = X_{\text{déc}} W^Q, \quad K = X_{\text{enc}} W^K, \quad V = X_{\text{enc}} W^V$$

- Quand le décodeur produit « student », il interroge l'encodeur → poids fort sur « étudiant ».
- C'est ce qui permet la **traduction** (alignement source ↔ cible).
- Pas de masque ici : l'encodeur a déjà tout vu.

Récap : 3 sous-couches

Masked self-attn → Cross-attn → FFN. Chacune entourée d'un Add & Norm.



Transformers : pourquoi ça change tout

RNN / LSTM (avant 2017)

- Traitement **séquentiel** token par token.
- Contexte lointain difficile à retenir.
- Non parallélisable → entraînement lent.

Transformer (depuis 2017)

- Toutes les positions vues **simultanément**.
- Attention globale → contexte long géré.
- Massivement parallélisable sur GPU.

Conséquence

Les Transformers rendent possible l'entraînement sur des **trillions de tokens** → c'est cela, et non un « génie algorithmique », qui a permis ChatGPT.

Prédiction token-par-token (auto-régressif)

Étape	Contexte & prédiction
1	[Le] [chat] → probabilités sur tout le vocabulaire → [est]
2	[Le] [chat] [est] → [sur]
3	[Le] [chat] [est] [sur] → [le]
4	[Le] [chat] [est] [sur] [le] → [tapis]
5	...jusqu'à un token [FIN] ou une limite de longueur.

- À chaque pas, le modèle produit une **distribution** sur $\sim 50\,000$ à $200\,000$ tokens.
- Il faut **choisir** un token parmi cette distribution → **stratégie de décodage**.
- Le token choisi est ré-injecté comme entrée pour l'étape suivante.

Stratégies de décodage : température, top- k , top- p

Paramètre	Effet	Quand l'utiliser
Greedy	Choisit toujours le token le plus probable.	Tâches déterministes (extraction, code).
Température T	$T < 1$: conservateur. $T > 1$: créatif. $T = 0$: $\approx$ greedy.	Ajuster selon le besoin factuel vs créatif.
Top- k	Garde les k tokens les plus probables, échantillonne parmi eux.	Couper la longue queue aléatoire.
Top- p (nucleus)	Garde les tokens dont la somme atteint p (ex: 0,9).	Standard moderne, s'adapte à chaque pas.

Règles pratiques

- Classification / extraction : `temperature=0`, décodage déterministe.
- Rédaction de rapport : `temperature=0.3--0.7`, `top_p=0.9`.
- **Jamais** de créativité sur un résultat qui doit être reproductible.

Fenêtre de contexte : une limite structurelle

- Un LLM ne peut traiter qu'un nombre **fini de tokens** à la fois : sa **fenêtre de contexte**.
- Inclut : prompt système + historique + document injecté + réponse en cours.
- Au-delà, il faut tronquer, résumer ou faire du RAG.

Modèle	Fenêtre (tokens)	Équivalent texte
GPT-3.5	16 k	≈ 25 pages
GPT-4o	128 k	≈ 200 pages
Claude 4.x	200 k – 1 M	≈ 300 à 1500 pages
Gemini 2.x	1 M – 2 M	≈ 1500 à 3000 pages
Llama 4	128 k – 1 M	Variable selon déploiement

Attention

Un grand contexte ne signifie pas une bonne utilisation du contexte (effet « lost in the middle »). Tester sur votre cas.

Limites structurelles à connaître

Hallucination

Le modèle peut produire un texte **fluide mais factuellement faux**. Il ne « sait » pas qu'il ne sait pas.

Mitigation : RAG, citation de sources, auto-vérification.

Coût & latence

Facturation au **token** (entrée + sortie).
Latence proportionnelle au nombre de tokens générés.

Mitigation : prompts courts, cache, modèle plus petit si possible.

Biais

Hérités des données d'entraînement (genre, origine, formulation).

Mitigation : prompts neutres, audit, validation humaine.

Confidentialité / RGPD

Données bancaires sensibles → API publique non contrôlée.

Mitigation : Azure OpenAI (données non conservées), chiffrement, DLP, masquage PII.

Récapitulatif : ce qu'un LLM *est* et *n'est pas*

Un LLM *est*

- Un modèle statistique qui prédit le prochain token.
- Un outil puissant pour transformer du texte.
- Excellent pour résumer, extraire, reformuler, classifier.
- Capable de raisonnement guidé (chain-of-thought).

Un LLM *n'est pas*

- Une base de connaissances fiable et à jour.
- Un système déterministe (par défaut).
- Un calculateur exact.
- Un juge de vérité.

Principe directeur

Traiter le LLM comme un **collaborateur junior très rapide** : capable, mais à encadrer, à vérifier, à équiper d'outils (RAG, appel de fonctions, flows Power Automate).

Objectifs du TP1 :

- Appeler un LLM via **API** (OpenAI / Azure OpenAI) ou via **HuggingFace Inference API**.
- Observer l'effet de `temperature`, `top_p`, `max_tokens`.
- Mesurer empiriquement la **fenêtre de contexte** et le **coût en tokens**.
- Reproduire un cas d'hallucination et une réponse correcte sur le même sujet.

Prérequis :

- Clé API (fournie) ou compte HuggingFace.
- Notebook Python ou environnement équivalent accessible depuis le navigateur.

Questions ?